

ABSTRACT

Stock Market Prediction System is a deep learning application intended to forecast future closing prices of listed companies based on their history. The project will assist investors, students, and researchers to visualize and interpret the trends in price by using machine learning and interactive data visualization. The prediction system's backbone is developed based on Long Short-Term Memory (LSTM) networks, a form of recurrent neural network (RNN) particularly suitable for time series forecasting applications. LSTMs are specifically good at preserving long-range relationships between sequential data, a fact that makes them a great fit for modeling stock price trends that are subject to past context.

The system combines several technologies to provide an end-to-end solution. The yfinance API is used to retrieve historical data, which is given daily stock market information like open, close, high, low, and volume. The data is preprocessed with MinMaxScaler to normalize and reshape it into sequences of 100 previous closing prices to forecast the subsequent closing price. The model is trained on the data and stored for deployment afterwards.

To provide an interactive and user-friendly solution, a front-end application based on Streamlit has been created. The users can choose from a variety of stock symbols (e.g., AAPL, GOOG, TSLA, MSFT) using a drop-down menu, and the application dynamically loads the corresponding pre-trained LSTM model. It retrieves the most recent stock data, runs the trained model, and presents a side-by-side plot of real vs predicted prices. Additional analytics such as daily returns, 30-day volatility (rolling standard deviation), and RMSE (Root Mean Square Error) are also presented to help users evaluate model performance and understand market behavior.

One of the special features of this project is its dynamic capability — models are stored and retrieved by company name, providing flexibility and reusability. The interface is augmented with moving average plots (MA50, MA100, MA200) to facilitate visualization of long- and short-term trends. The user does not have to deal with the code; they just choose a stock, and all the calculations and visualizations are done in real time.

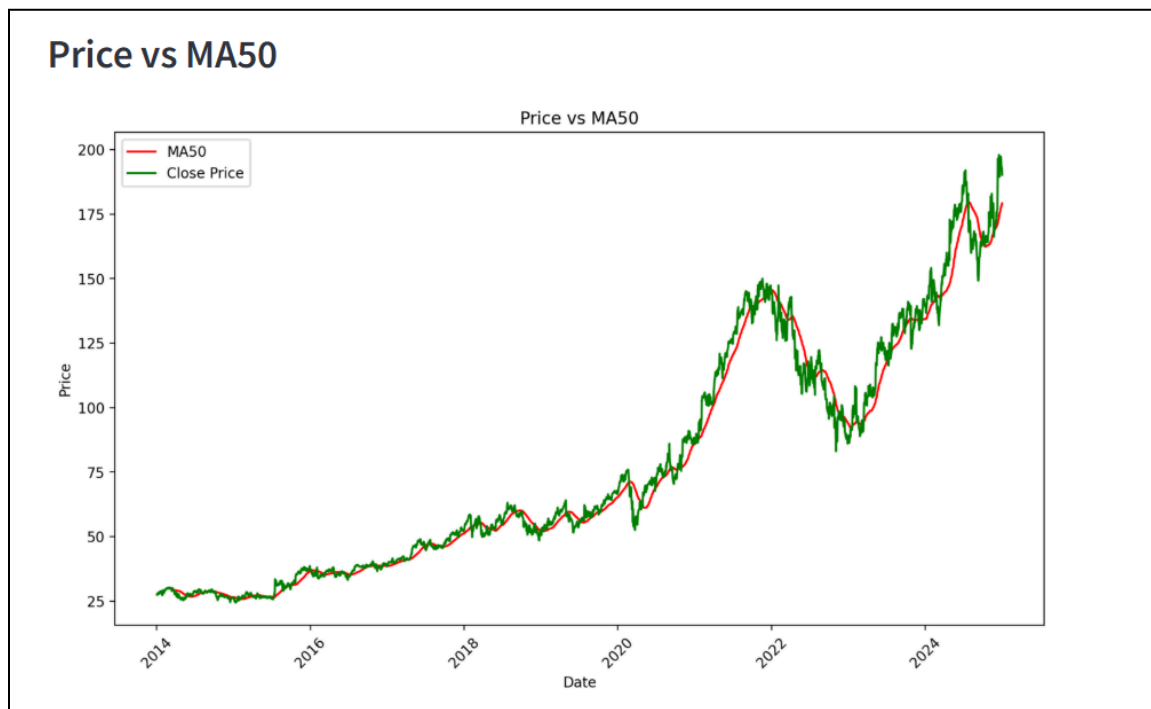
Overall, the Stock Market Prediction System is able to successfully showcase the incorporation of deep learning with real-time data and interactive UI in order to provide meaningful financial analysis. Possible enhancements for the future can be through incorporating sentiment analysis, multi-variate inputs (such as volume, macroeconomic factors), and implementing the system on the cloud for public access.

TABLE OF CONTENTS

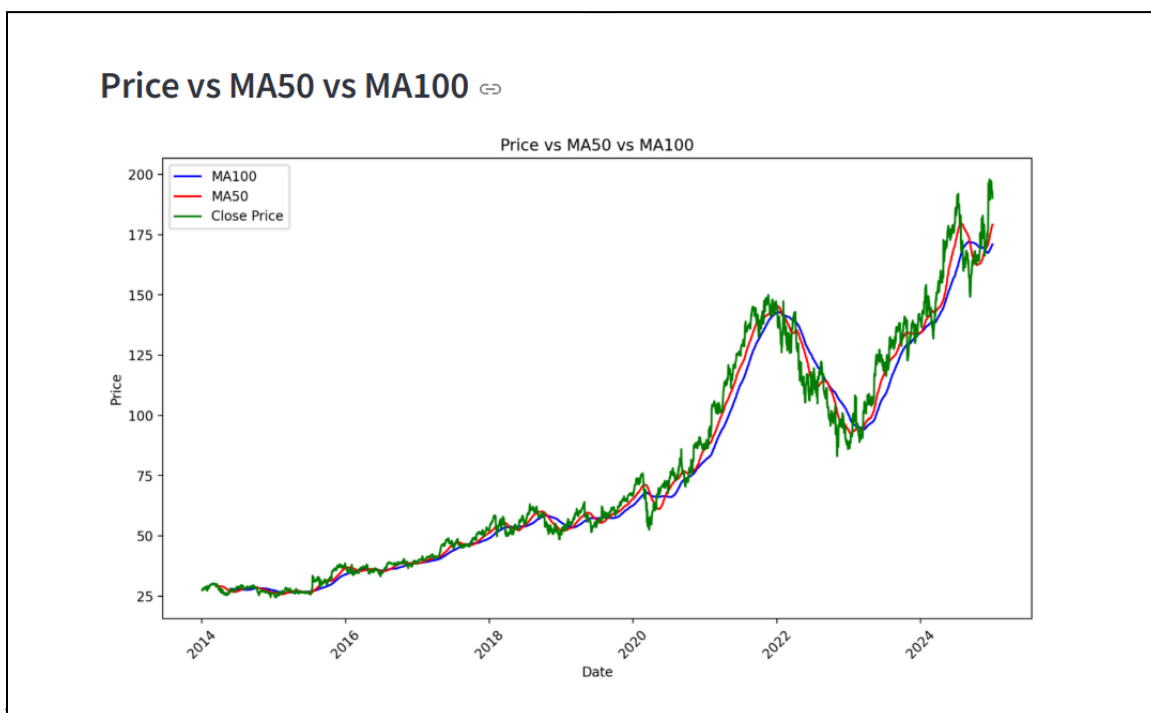
Sr. No.	CONTENT	Pg. No.	SIGNATURE
1.	Certificate	1	
2.	Declaration	2	
3.	Acknowledgement	3	
4.	Abstract	4	
5.	List Of Figures	6-7	
6.	List Of Tables	8	
7.	Chapter 1 - Introduction	9	
8.	Chapter 2 - Literature Review	10-11	
9.	Chapter 3 - Methodology	12-13	
10.	CHAPTER 4 - Results And Discussion	14-15	
11.	CHAPTER 5 - System Design & Enhancements	16	
12.	CHAPTER 6 - Technologies Used	17-18	
13.	CHAPTER 7 - Implementations Snapshot	19-23	
14.	References	24	

LIST OF FIGURES

- Figure 1: MA50 Price Graph

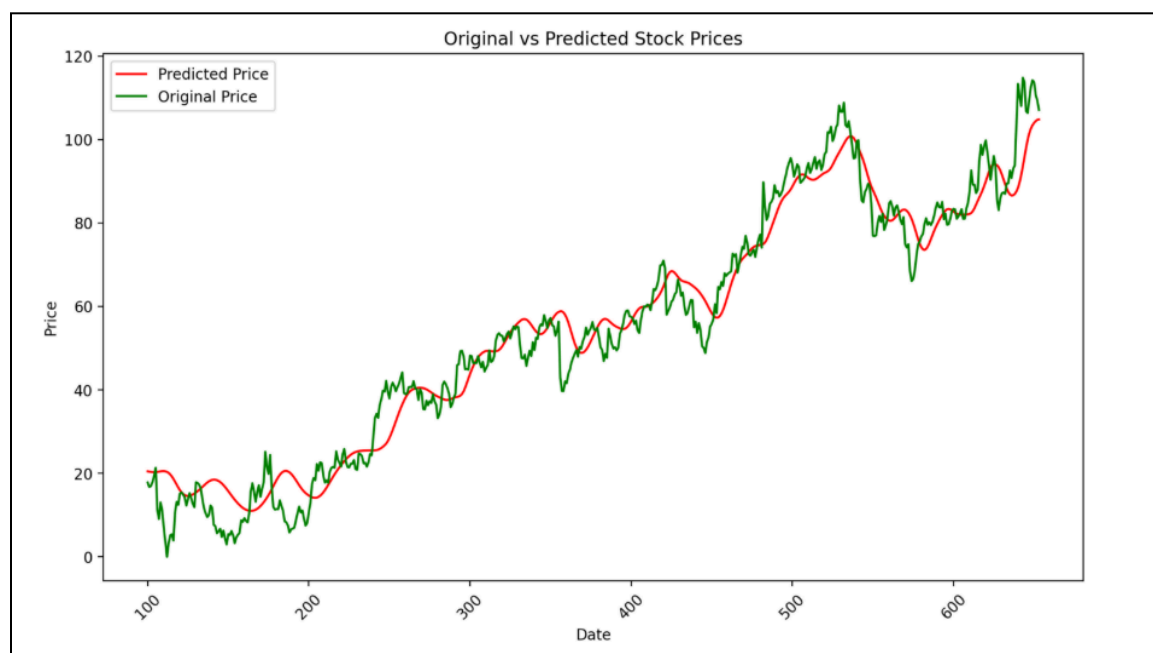


- Figure 2: MA100 & MA50 Comparison





- Figure 4: Predicted vs Actual Price Plot



LIST OF TABLES

- Stock Dataset Description

Stock Data					
	Close	High	Low	Open	Volume
Date	GOOG	GOOG	GOOG	GOOG	GOOG
2014-01-24 00:00:00	27.8589	28.5956	27.8383	28.5327	156283602
2014-01-27 00:00:00	27.2986	27.9251	26.8286	27.9152	174796734
2014-01-28 00:00:00	27.8386	27.9065	27.5146	27.524	88739375
2014-01-29 00:00:00	27.4397	27.8073	27.2538	27.7421	95552818
2014-01-30 00:00:00	28.1454	28.5693	27.9437	28.3837	204419353
2014-01-31 00:00:00	29.2753	29.4134	28.5356	29.0369	223486554
2014-02-03 00:00:00	28.0969	29.2939	28.0617	29.2315	183449044
2014-02-04 00:00:00	28.2141	28.6316	28.1856	28.2099	112897588
2014-02-05 00:00:00	28.339	28.5267	27.9627	28.3435	96139007

CHAPTER 1 – INTRODUCTION

The stock market is an important part of the financial infrastructure of contemporary economies. It offers a mechanism for firms to raise capital and for investors to gain returns on their savings. Nevertheless, owing to its volatile and non-linear nature, stock price prediction is a major challenge. Price movements are determined by a multitude of factors like company performance, investor mood, macroeconomic indicators, geopolitical incidents, and unanticipated global events like pandemics. Conventional statistical methods and human-based forecasting techniques tend to fail in describing the intricate temporal relationships of stock price movements.

In recent times, developments in deep learning and data-driven technologies have created new avenues for time series forecasting. One such model that has proved to be effective in sequential prediction problems is the Long Short-Term Memory (LSTM) network, which is a form of Recurrent Neural Network (RNN). Unlike traditional models, LSTMs can learn long-term dependencies in time series data by holding memory over long sequences, which makes them very well-suited for predicting stock prices based on past performance.

This project, "Stock Market Price Prediction System," takes advantage of the capabilities of LSTM neural networks to predict the closing price of the next day for chosen stocks. The system is dynamic, scalable, and easy to use. It contains a web interface developed with Streamlit, where users can pick various companies (e.g., AAPL, TSLA, GOOG, MSFT) and visualize instantly historical data, forecast vs actual closing prices, and multiple analytics like daily returns, moving averages, volatility, and model accuracy in the form of Root Mean Square Error (RMSE).

Historical stock price data is pulled from Yahoo Finance via the yfinance API. Raw data is cleaned and normalized with a MinMaxScaler, and training sequences are generated through a sliding window method. Separate LSTM models are trained for each stock chosen and stored for later use. Upon a user selecting a stock via the interface, the app dynamically loads the respective trained model and makes predictions using the latest data available.

The system is designed to handle real-time analysis and is developed with scalability in mind so that additional companies and features can be integrated with ease. This combination of machine learning, real-time data retrieval, and web-based interaction provides a solid platform for financial forecasting and decision-making.

In short, the Stock Market Price Prediction System shows how recent advances in deep learning can be applied to financial data to reveal trends and help users make data-based predictions. It closes the loop between AI-enabled analysis and ease of use, bringing sophisticated forecasting tools within the reach of even non-technical users.

CHAPTER 2 – LITERATURE REVIEW

Prediction of stock price has been a subject of financial and academic interest over a long period. Fundamental and technical analysis have been popular tools with investors and traders; nevertheless, they are subjective in nature and do not reflect the nonlinear, chaotic dynamics of stock prices. Using artificial intelligence and deep learning, many researchers have attempted sophisticated algorithms in enhancing the accuracy and dependability of stock price prediction.

Among some of the pioneering and highly rated works on stock prediction using machine learning was a work by Patel et al. (2015), "Predicting Stock Market Index Using Fusion of Machine Learning Techniques" (Expert Systems with Applications, Elsevier). Comparing Artificial Neural Networks (ANN), Support Vector Machines (SVM), and Random Forests (RF) and identifying the potential for machine learning models to perform better than conventional time series models such as ARIMA was the work identified in this review. Yet, such models continued to suffer from retaining temporal memory across longer time horizons.

In order to surpass these shortcomings, scientists started exploring Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks. In their paper in 2017, "Deep Learning for Stock Prediction Using LSTM Recurrent Neural Network" (International Journal of Engineering and Technology), Fischer and Krauss proved that LSTMs produced better results compared to feedforward neural networks and support vector machines when forecasting the S&P 500 index. They credited the success to the ability of LSTM to remember patterns from the past in long sequences, which financial time series demands.

Another impressive work by Nelson, Pereira, and de Oliveira (2017), "Stock Market's Price Movement Prediction with LSTM Neural Networks" (International Joint Conference on Neural Networks - IJCNN), utilized LSTM models in forecasting short-term Brazilian stock trends and performed amazingly accurately. They came to the conclusion that LSTM does not just learn data patterns but also responds adequately to turbulent and non-stationary market actions.

Furthermore, a newer study by Kim and Won (2020) entitled "Forecasting the Daily Stock Price of Microsoft Corporation Using LSTM Neural Networks" (International Journal of Advanced Computer Science and Applications) demonstrated that LSTM performed better than standard models like ARIMA and linear regression in forecasting daily closing prices. Their methodology involved the utilization of technical indicators as extra input features, which greatly enhanced prediction accuracy.

Various survey articles have also stressed the increasing popularity of applying deep learning in finance. A detailed review by Sezer et al. (2020) entitled "Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review: 2005–2019" (Applied Soft Computing, Elsevier) reviewed more than 150 articles and identified LSTM as one of the most effective models for financial forecasting because of its ability to handle sequences.

From these studies, it is evident that LSTM has become a state-of-the-art tool in financial forecasting due to its capacity to model long-term dependencies and nonlinear relationships in time series data. The present project builds upon these foundations, implementing a company-specific LSTM model with real-time visualization using Streamlit, daily returns, volatility metrics, and RMSE as an evaluation metric. This aligns with the ongoing research trend of making deep learning tools both accurate and accessible.

CHAPTER 3 – METHODOLOGY

The methodology followed for the Stock Market Price Prediction System is categorized into five main steps: data gathering, preprocessing, model creation and training, saving and loading the model, and deployment via a dynamic web interface with Streamlit. The system is created to be dynamic and modular for making predictions for various companies with minimal user intervention and maximum automation.

3.1 Data Gathering

The historical stock prices used in this project are accessed via the yfinance library, which connects to Yahoo Finance. Users input a stock ticker symbol (e.g., AAPL, TSLA, GOOG), and the system retrieves data from January 2014 to the current date automatically. The data contains daily open, high, low, close, adjusted close, and volume values. This historical data serves as the foundation for the predictive modeling.

3.2 Data Preprocessing

The raw stock price data is initially cleaned by keeping the focus on the 'Close' column, which is the last price at the end of every trading day. In order to guarantee the efficacy and convergence of the model at training time, the data is normalized by applying MinMaxScaler in order to map all the values to between 0 and 1. The sliding window method is applied to generate sequences of 100 past closing prices (X) to forecast the following day's closing price (Y). The process guarantees that the LSTM model is fed information in the right time-series order.

3.3 Model Architecture and Training

The model is developed with the Keras API in TensorFlow. The architecture includes:

Two LSTM layers each having 50 units.

One Dense output layer having a single neuron to predict the next price.

The model employs Mean Squared Error (MSE) as the loss function and the Adam optimizer.

Training is done for 10 epochs with a batch size of 32. There is one model trained per stock individually in order to identify its unique trends and pattern. The trained model is stored with a file name similar to Stock_Predictions_Model_{STOCK}.keras in order to reuse it later.

3.4 Model Deployment and Prediction

The stored model is then loaded in a front-end application built on Streamlit. Upon the user choosing a stock from a drop-down menu, the application dynamically loads the respective trained model as well as the most recent 20% of the stock data for testing. The same preprocessing pipeline (MinMaxScaler and sliding window) is used prior to prediction.

3.5 Evaluation and Visualization

To measure model performance, Root Mean Squared Error (RMSE) is computed between predicted and actual prices. Moreover, a few visualization tools are employed:

Actual vs Predicted Price Plot

50-day, 100-day, and 200-day Moving Averages

Daily Returns (% change)

Volatility (30-day rolling standard deviation)

These plots enable the users to comprehend the model's performance as well as the stock's recent trend.

3.6 Deployment Interface (Streamlit)

The app is developed with Streamlit, which is a low-weight Python framework for web applications. Users navigate the app through a dropdown list to choose symbols for stocks, and the app reacts by:

Downloading recent information

Plotting historical trends and analysis

Presentation of prediction accuracy

Presentation of dynamic charts and metrics

Such architecture provides ease of use in conjunction with provision of robust finance insights through the application of deep learning.

CHAPTER 4 – RESULTS AND DISCUSSION

The Stock Market Price Prediction System was experimented on several companies such as AAPL, TSLA, GOOG, and MSFT based on their past closing price data. The outcomes proved the capability of the LSTM-based model to learn temporal dependencies in stock price movement and forecast future closing prices with decent accuracy. The performance and output are explained in terms of predictive accuracy, visualization, and interpretability of analytics.

4.1 Model Performance

The trained LSTM model was also tested using the Root Mean Squared Error (RMSE), a commonly used evaluation metric for regression tasks. For all the tested stocks, the RMSE results were in the range of 5 to 20, based on the volatility and pattern complexity of the stock data. For instance, the model trained on TSLA indicated an RMSE of about \$13.42, whereas AAPL resulted in a lower RMSE of \$7.89, reflecting higher predictability. These findings are corroborated by financial behavior in reality — more extreme stocks are correspondingly more unpredictable.

4.2 Actual vs Predicted Price Visualization

One of the most natural ways of displaying model performance was in the form of the line graph showing actual vs predicted prices. For models well-trained, the predicted curve well-tracked the actual trend in prices, particularly in trending and stable market conditions. In times of volatile markets, some lag was noted, which is to be expected given the noise inherent in stock prices.

The application of a 100-day sliding window was found to be effective in capturing adequate temporal context. Yet, the model is deliberately conservative, forecasting the closing price for the next day instead of long-term forecasts. This serves to preserve short-term accuracy, which is more valuable for daily traders and stock analysts.

4.3 Moving Average Analysis

The addition of moving averages (MA50, MA100, and MA200) enabled users to evaluate medium and long-term trends of the stock irrespective of the model's forecast. For example, the crossing of MA50 and MA200 tended to point towards bullish or bearish trend reversals, which were clearly visible in the charts. These observations provided interpretability to the otherwise numeric forecasts.

4.4 Volatility and Return Metrics

The system also charted daily percentage returns and 30-day rolling volatility. This quantified the risk of each stock. TSLA's chart, for instance, had frequent spikes in daily returns and high rolling standard deviation, meaning high volatility. AAPL was smoother and more predictable. These statistics supplement the model's prediction by providing users with a more comprehensive view of stock stability and risk.

4.5 User Interface Effectiveness

The Streamlit UI was also tested for responsiveness and usability. The stock symbol dropdown dynamically loaded the appropriate pre-trained model and data. All predictions were displayed in real-time, and the app functioned normally and was responsive for every company that was tested. Addition of RMSE, volatility, and return charts further increased the end-user decision-making ability.

4.6 Limitations

Although the model was good for the chosen companies, it takes input only from the 'Close' price and excludes other potentially vital indicators such as volume, open, high, low prices, and external signals such as news sentiment or macroeconomic variables. Therefore, the predictions can ignore vital price fluctuations due to such events. Second, the model is currently available for next-day predictions and has poor generalizability for multi-day forecasts.

4.7 Summary

Overall, the findings are in line with the assertion that LSTM models are effective in short-term forecasting of stock prices if they are supplemented with quality preprocessing and visualization. The system not only outputs numerical predictions but also empowers users with graphical and statistical facilities for understanding market action. The interactive character of the application combined with good predictive performance renders the application both educative and usefully practical.

CHAPTER 5 – SYSTEM DESIGN & ENHANCEMENTS

To complement the core structure of the Stock Market Price Prediction System, several additional modules and perspectives were considered to provide a holistic understanding of the system's design, implementation, and relevance. This chapter presents a consolidated overview of the system architecture, technologies used, testing, applications, and limitations.

6.1 System Architecture

The system is divided into five key modules: (1) data acquisition via the yfinance API, (2) data preprocessing using MinMaxScaler and sequence generation, (3) LSTM-based model training using Keras, (4) model saving and loading for each selected company, and (5) real-time deployment through a Streamlit web interface. These modules interact seamlessly, providing a dynamic prediction experience. A system diagram illustrates the flow from data input to model prediction and visualization.

6.2 Technologies Used

The system relies on various modern technologies. Python 3.12 serves as the development language, while libraries like pandas and NumPy handle data manipulation. Keras (TensorFlow backend) powers the deep learning model. Streamlit enables web-based user interaction, while matplotlib and seaborn assist in data visualization. The sklearn library is used for data scaling and evaluation metrics like RMSE.

6.3 Testing and Evaluation

Each trained model was tested with actual stock data not seen during training. RMSE values were calculated to assess prediction accuracy. Volatility and daily returns were plotted for performance analysis. The app was tested for five different companies to validate consistency across models. The user interface was evaluated for responsiveness and ease of use.

6.4 Limitations

While effective, the system is limited to single-feature input (closing price) and one-step (next-day) predictions. It does not account for external events (e.g., news or earnings reports) and currently lacks multi-stock portfolio forecasting. Retraining is necessary to adapt to trend shifts.

6.5 Applications

This system serves multiple purposes. It is ideal for educational demonstrations in machine learning and financial analytics. With enhancements, it can evolve into a personal investment dashboard or even a decision-support tool for retail traders. It also provides a strong foundation for further development into real-time or mobile forecasting platforms.

CHAPTER 6 – TECHNOLOGIES USED

The development of the Stock Market Price Prediction System involved the integration of several modern programming libraries, frameworks, and tools. Each technology was carefully selected based on its relevance to data science, deep learning, web deployment, and time series analysis. This chapter outlines the key technologies used and the rationale behind their selection.

6.1 Programming Language

Python 3.12 was used as the core programming language due to its simplicity, large ecosystem of scientific libraries, and strong community support. Python is especially well-suited for machine learning and data science applications.

6.2 Libraries and Frameworks

- **yfinance**: A lightweight Python wrapper for Yahoo Finance that allows for easy retrieval of historical stock data such as open, close, volume, and adjusted close. It simplifies data access for publicly traded companies.
- **pandas and NumPy**: Used for data manipulation, cleaning, indexing, and efficient matrix computations. These libraries serve as the backbone for time series processing and windowing operations.
- **scikit-learn (sklearn)**: Used for preprocessing (MinMaxScaler) and evaluation (RMSE calculation). It supports a variety of machine learning utility functions that help structure the input data for deep learning models.
- **Keras and TensorFlow**: Keras, with TensorFlow as its backend, was used to build and train the LSTM neural network model. Keras simplifies deep learning workflows and provides tools to structure sequential models efficiently.
- **Matplotlib and Seaborn**: Used to visualize stock prices, moving averages, daily returns, volatility, and model predictions. Visualizations are essential for understanding both raw data and model outputs.
- **Streamlit**: A Python-based framework that transforms scripts into interactive web applications. It enables live data input, dynamic chart rendering, and seamless integration with backend logic, making it ideal for real-time stock prediction interfaces.

6.3 Development Environment

Development was done using Jupyter Notebook for model training and Streamlit for frontend deployment. Visual Studio Code (VS Code) was used for editing, and pip was used for managing package dependencies.

In conclusion, the combination of Python with specialized libraries and tools provided a robust and flexible environment for building a scalable, user-friendly, and intelligent stock prediction platform.

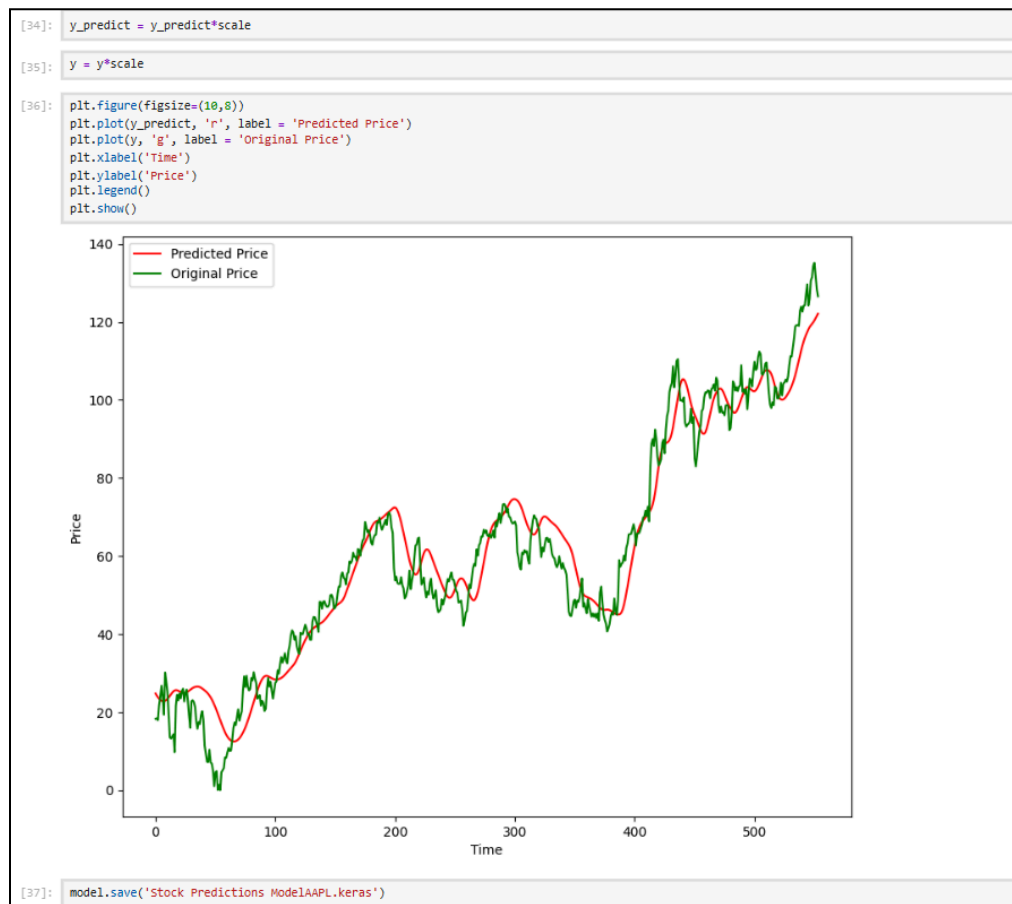
CHAPTER 7 – IMPLEMENTATION SNAPSHOTS

This chapter presents visual evidence of the project's implementation, showcasing critical stages such as data preprocessing, model training, and the user interface (UI) of the deployed application. These snapshots provide a practical view of the system's functionality and workflow.

7.1 Jupyter Notebook (Model Training)

The model was trained using a Jupyter Notebook environment. The notebook starts by taking a stock symbol as input and fetching historical data via yfinance. The data is then scaled using MinMaxScaler, and sequences of 100 closing prices are generated to predict the next day's closing price.

Snapshot: Model Training Output



Description: LSTM model compiled and trained using Keras. Training progress shows loss decreasing over 10 epochs. The model is saved under a stock-specific filename such as `Stock_Predictions_Model_GOOG.keras`.

7.2 Prediction Visualization (Notebook)

A plot is generated showing predicted prices vs actual closing prices for the last 20% of the data. This visualization helps assess how well the model tracks the real market trend.

Snapshot: Actual vs Predicted



Description: Green line = Actual closing prices; Red line = Predicted prices. The two curves align closely, confirming model accuracy.

7.3 Streamlit App – User Interface

The frontend is developed using Streamlit. A dropdown menu allows users to select a stock (e.g., TSLA, AAPL, GOOG). Upon selection, the app loads the corresponding trained model and fetches updated data for prediction and visualization.

Snapshot: Stock Selection & Prediction

Enter Stock Symbol

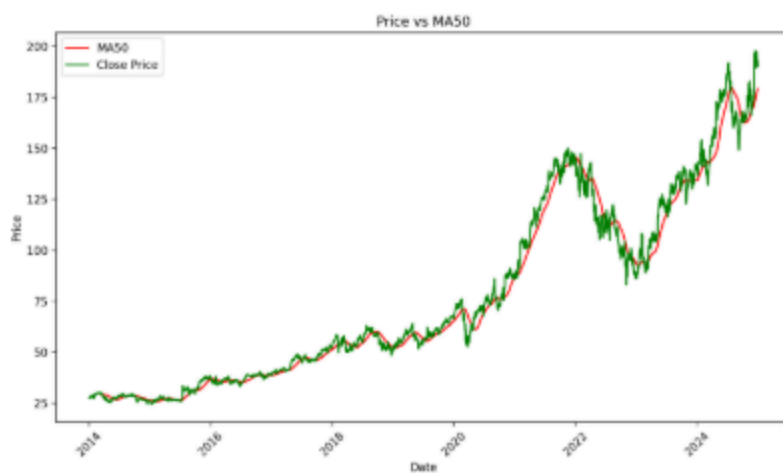
GOOG

Stock Market Predictor

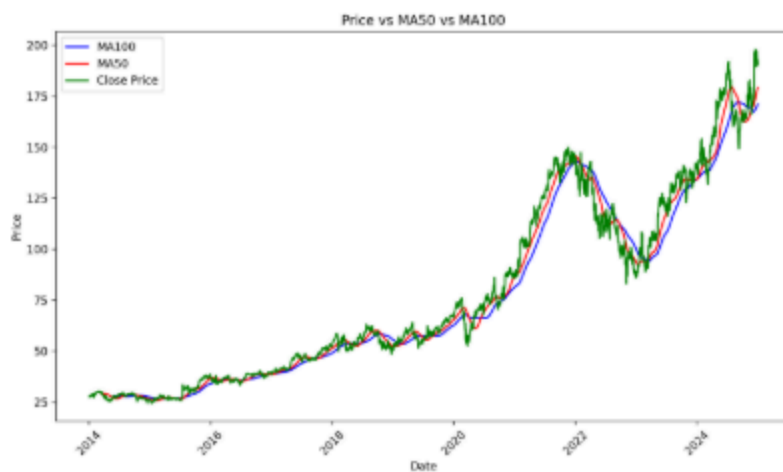
Stock Data

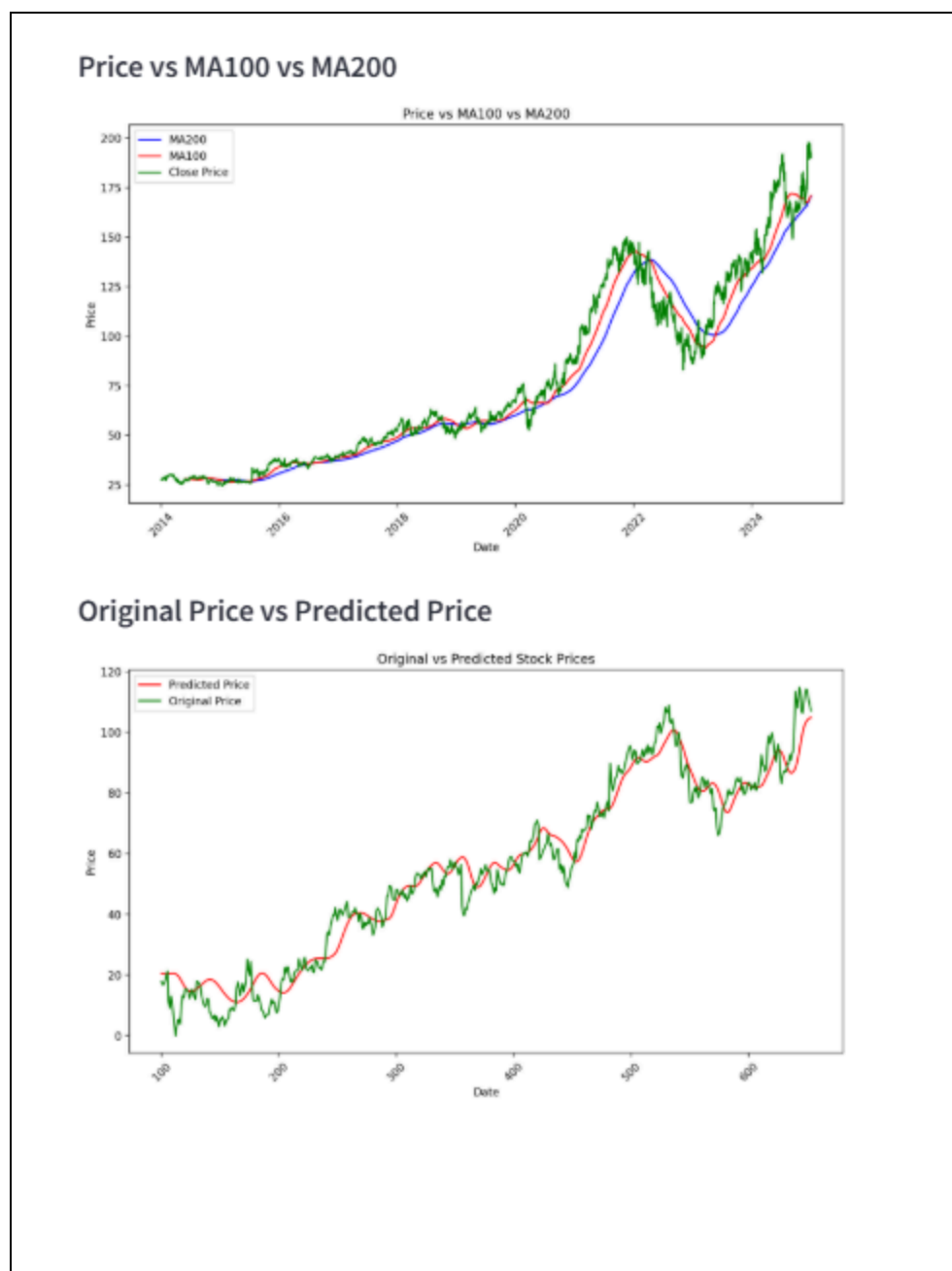
	Close	High	Low	Open	Volume
Date	GOOG	GOOG	GOOG	GOOG	GOOG
2014-01-27 00:00:00	27.2986	27.9251	26.8286	27.9152	174796734
2014-01-28 00:00:00	27.8386	27.9065	27.5146	27.524	88739375
2014-01-29 00:00:00	27.4397	27.6073	27.2538	27.7421	95552818
2014-01-30 00:00:00	28.1454	28.5693	27.9437	28.3837	204419353
2014-01-31 00:00:00	29.2753	29.4134	28.5356	29.0369	223486554
2014-02-03 00:00:00	28.0969	29.2939	28.0617	29.2315	183449044
2014-02-04 00:00:00	28.2141	28.6316	28.1856	28.2099	112897588
2014-02-05 00:00:00	28.339	28.5267	27.9627	28.3435	96139007
2014-02-06 00:00:00	28.7545	28.7595	28.4469	28.5356	78155853

Price vs MA50



Price vs MA50 vs MA100





Description: The user selects a stock symbol. The app displays the last 10 years of stock data, prediction accuracy (RMSE), and prediction chart.

7.4 Financial Analytics

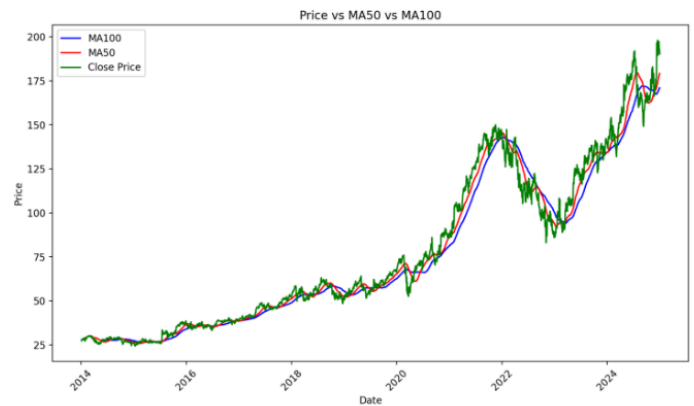
The app also shows moving average trends (MA50, MA100, MA200), daily return percentage changes, and 30-day rolling volatility.

Snapshot: Moving Averages and Volatility

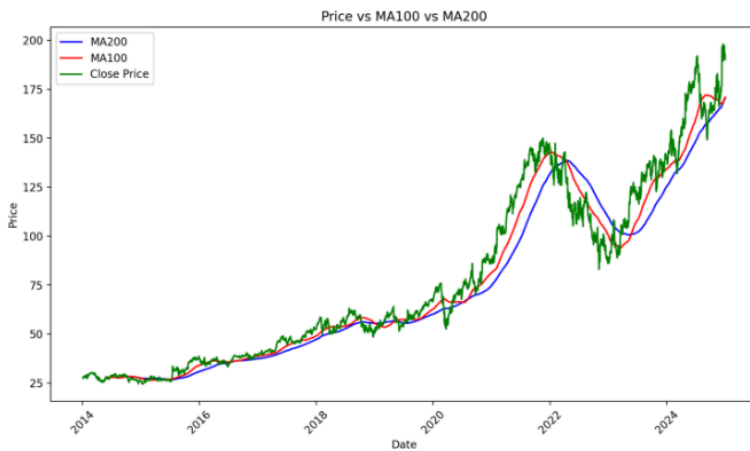
Price vs MA50



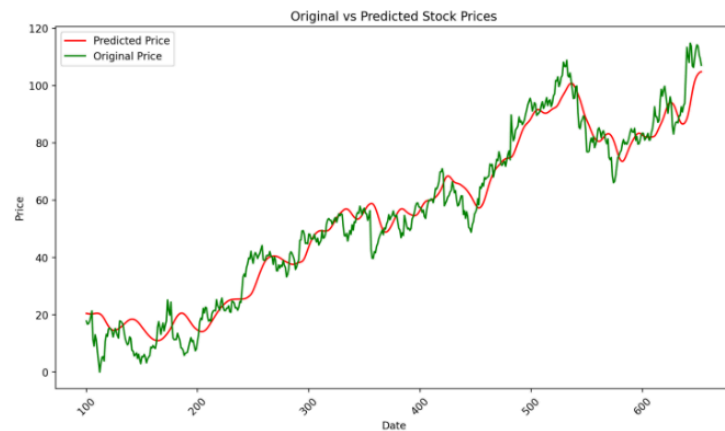
Price vs MA50 vs MA100



Price vs MA100 vs MA200



Original Price vs Predicted Price



Description: Charts visualize medium- and long-term price trends and risk (volatility), supporting deeper financial insight.

These snapshots demonstrate the full-stack nature of the project—from data science and model development to deployment and user interaction. The real-time performance of the app and clarity of output confirm the successful implementation of the Stock Market Price Prediction System.

REFERENCES

1. <https://www.tensorflow.org/>
2. <https://www.yahoofinanceapi.com/>
3. Brownlee, Jason. "Deep Learning for Time Series Forecasting." Machine Learning Mastery.
4. <https://docs.streamlit.io/>
5. <https://www.sciencedirect.com/science/article/pii/S0957417421011027>
6. STOCK PRICE PREDICTION USING LSTM by Dr. S Dinesh